

```

Return 0 if no simplifications are possible. */
/*
simplify_subreg (enum machine_mode outermode, rtx op,
enum machine_mode innermode, unsigned int byte)
{
/* Little bit of sanity checking. */
gcc_assert (innermode != VOIDmode);
gcc_assert (outermode != VOIDmode);
gcc_assert (innermode != BLKmode);
gcc_assert (outermode != BLKmode);

gcc_assert (GET_MODE (op) == innermode
|| GET_MODE (op) == VOIDmode);

gcc_assert (byte % GET_MODE_SIZE (outermode) == 0);
gcc_assert (byte < GET_MODE_SIZE (innermode));

if (outermode == innermode && !byte)
return op;

"simplify-rtx.c" [readonly] 3875 lines, 17398 characters

```

Figure 3: Definition of the called function

```

target = expand_divmod (0, TRUNC_DIV_EXPR, Pmode, target,
NULL_RTX, 1);
target = expand_mult (Pmode, target,
GCC_INT (required_align / BITS_PER_UNIT),
NULL_RTX, 1);
}

/* Now that we've committed to a return value, mark its alignment. */
next_real_pointer (target, required_align);

/* Record the new stack level for nonlocal gotos. */
if (Cfun->nonlocal_goto_save_area != 0)
update_nonlocal_goto_save_area (C);

return target;

/* A front end may want to override GCC's stack checking by providing a
runtime routine to call to check the stack, so provide a mechanism for
explore.c" [readonly] 1822 lines, 38728 characters

```

Figure 4: Call to the specified function

from the Cscope terminal.

To find the functions called by a particular function, again, enter the name of that function against the option. (I tried to find the functions called by the function 'assemble_integer'.) The results get listed as before; navigate them as was done earlier to go to the particular file containing the global definition of the calling function, at the point where the chosen function is called (shown in Figure 2). Then go to the called function name and use **CTRL+J** to jump to the definition of the called function, as shown in Figure 3.

To find functions calling a function, enter the name of the function; I searched for 'expand_mult'. As before, choose one of the results to open the file containing that function definition, as in Figure 4. Then go to the called function name and use **CTRL+J** to jump to the definition of the called function, as shown in Figure 5.

CTRL+T is used to trace back to where you started your search in the particular directory.

To find a particular text string (for example, 'Print an error'), follow the now-familiar procedure to open the file containing that text string, as shown in Figure 6.

Ctags vs Cscope

From the previous article and this, you can see that both tools are helpful in browsing through source code. You might be wondering which one to use. There are some basic differences. You might have noticed that Ctags takes you to the function where it is defined,

```

you should swap the two operands if GPR would be constant. */
/*
expand_mult (enum machine_mode, rtx op0, rtx op1, rtx target,
int unsigned)
{
enum mult_variant variant;
struct algorithm_algorithm;
int max_cost;
bool speed = optimize_for_speed_0;

/* Handling const0_rtx here allows us to use zero as a regval for
coeff below. */
if (op0 == const0_rtx)
return const0_rtx;
if (op1 == const0_rtx)
return op0;
if (op0 == const0_rtx)
return expand_mult (mode, GET_MODE_CLASS (mode) == MODE_INT
&& unsignedp 64 Flag_trap
"expand.c" [readonly] 3729 lines, 18813 characters

```

Figure 5: Global definition of the called function

```

2007-06-12 Seongbae Park <seongbae.park@gmail.com>
/* opts.c (common_handle_option): Handle new option -fdbg-ctrl-list.
* dbgcpt (<dbg-ctrl-list-init-by-name>): Return value
to indicate an error.
* dbg-ctrl-process_single_ptr (<dbg-ctrl-list-all_counters>):
new functions.
* dbg-ctrl-process_opt (<dbg-ctrl-list-all_counters>):
* dbgcpt (<dbg-ctrl-list-init-by-name>): New function declaration.
* common_opt (<fdbg-ctrl-list>): New.
* dbg-ctrl (<fdbg-ctrl-list, -fdbg-ctrl>): New.

2007-06-12 Eric Botcazou <eribotcazou@adacore.com>
* tree-ssa-align.c (finalize_ref_all_pointers): Clear pt_anything
flag on ref-all pointers.

2007-06-12 Andrew Pinski <andrew.pinski@playstation.sony.com>
/* middle-end/31579
* expr.c (expand_expr_addr_expr_3): Call expand_expr
for the offset with the modifier as EXPAND_INITIALIZER
if the modifier is EXPAND_INITIALIZER.
* expand_expr_real_3 <case INTEGER_CST>: Don't force to
a register if we had an overflow.
"Changelog-2007" [readonly] 84178 lines, 1349622 characters

```

Figure 6: Location of the text string

whereas Cscope will show you the place where the function is defined, as well as the places where it is called (or referenced). Another important difference is that Ctags offers auto-completion of code. For example, if my_structure is a structure object, and you type my_structure. or my_structure->, it will pop up the list of elements of the structure. So, whether to use Ctags or Cscope is your choice; I generally use a combination of both, depending on the requirement! 

References and further reading

- [1] http://vim.wikia.com/wiki/Browsing_programs_with_tags
- [2] <http://www.fsl.cs.sunysb.edu/~rick/cscope.html>
- [3] http://cscope.sourceforge.net/cscope_vim_tutorial.html

By: Gaganpreet Kaur

The author is a software engineer at RF-Silicon Technologies Pvt Ltd. She is currently working as a part of the GCC Compiler Team on machine descriptions specific to targets. She can be reached at kang_276@yahoo.in.

Edited and mentored by: Deepti Sharma, project manager, Software Group, RF-Silicon Pvt Ltd. Deepti has considerable experience in design, development and quality validation of system software tools (compilers, assemblers, device drivers, etc) for embedded systems. She has also been a mentor and trainer on compiler development, following good practices and maintaining quality. She loves writing for and can be reached at deepti.acmet@gmail.com.